Remedy

MCU Bootloader & Apploader

for Microchip PIC32MX7xx on SD/MMC Card

Installation and User's Guide

Remedy
MCU Bootloader & Apploader
for Microchip PIC32MX7xx on SD/MMC Card

RoweBots Research Inc.
119 Church St.
Kitchener/Waterloo, Ontario N2G 2S3
Canada

Technical Support
Web: www.rowebots.com/support

This manual is, in its entirety, copyright © RoweBots Research Inc. 1994-2013 and protected by international copyright laws.

All rights are reserved. No part of this book may be reproduced in any form, except brief excerpts for the purpose of review, without the written permission of RoweBots Research Inc.

All trademarks are the property of their respective owners.

1. Introduction 1

2. Overview 1

2.1 Bootloader and UserApp architecture 1

2.1.1 User Application 2

2.1.2 Application size 2

3. Quick start bootloader and user application 4

3.1 Download an application by bootloader 4

4. Algorithm and modes of bootloader operation 6

4.1 Bootloader modes. 6

4.1.1 Simple firmware update mode 6

4.1.2 Recovery firmware update mode 7

5. Application update example 10

5.1 Preparing firmware binary files 10

5.2 Programming binary files 10

6. Firmware encryption / decryption 13

7. Watchdog Timer 14

8. Device configuration bits 14

9. Appendix How to create a user application demo for bootloader 15

Figure f1

Memory Map 1

Figure f2

Bootloader starting screen 4

Figure f3

Prepare *.ocf file for bootloader 4

Figure f4

Write *.ocf file to SD card 5

Figure f5

Rename *.ocf file to fw.ocf 5

Figure f6

Flash new firmware. Run new user application 5

Figure f7

Bootloader Flow diagram without recovery mode 7

Figure f8

Bootloader Flow diagram with recovery mode 8

Figure f9

Application Flow diagram with recovery mode 9

Figure f10

Telnet user application session 12

Figure f11

The PC “Remedy File Coder” application 13

Figure f12

Mapping 128 bits key encryption with keys of panel “Remedy File Coder” application 13

Figure f13

Compliance with the keys in the program prescribed by“Remedy File Coder” with the keys in the bootloader . . .13

Figure f14

Project Settings 15

Figure f15

Add library files which 16

Figure f16

Set Includes path and Preprocessor macros. 16

1. Introduction

This document describes how to use MCU Bootloader provided by RoweBots Research Inc. This program allows to update software on the MCU by writing the firmware on SD card. Supported file systems are FAT, FAT16, and FAT32. A optional debug information is displayed via UART1 (115200,N,1).

2. Overview

The bootloader is a small piece of software that allows the user to download an update of code into flash memory. After every power up or reset for the MCU, the bootloader will check the SD card for new firmware with name fw.ocf to program into the FLASH memory.

It is possible to write new firmware file to SD card directly from PC (SD card should be formatted with FAT file system). It is also possible to remotely write this firmware using tftp. An example application will be described below.

The default path for the new firmware file is defined in "bootloader.h" for bootloader application:

```
FIRMWARE_DIR + FIRMWARE_NAME (default path: firmware/fw.ocf)
```

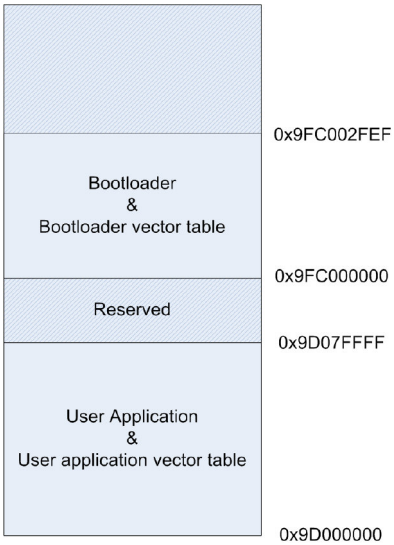
2.1 Bootloader and UserApp architecture

The bootloader is placed into the boot section of chip flash.

The User application can be placed into program flash memory.

The following figure shows the memory map with the Bootloader and UserApp memory footprint for PIC32MX795F512L chip.

Figure f1
Memory Map



2.1.1 User Application

User Application must define a special structure "**app_header**" (with size 32 bytes), placed at the end of application flash memory , which contain information about application starting point and some other important fields. All fields will be described later in this document. This structure should be placed as a section at address 0x9D07FFD0 (for PIC32MX795F512L). This address is set in a file app_32MX795F512L.ld:

```
MEMORY
{
  kseg0_program_mem    (rx)  : ORIGIN = 0x9D001490,
  LENGTH = (0x80000 - 0x1490 - 0x40)
  …
  app_hdr_mem                : ORIGIN = 0x9D07FFD0,
  LENGTH = 0x30
  …
  configsfrs                 : ORIGIN = 0xBFC02FF0,
  LENGTH = 0x10
}
```

*Important **note:***

The user should not modify placement address for this structure, because the Bootloader is expecting only this address for structure "app_header".

2.1.2 Application size

The maximum or max application size depends on the MCU type. Users can specify this size depending on their needs, so the bootloader will only program part of flash. This may be done by specifying starting point for application in structure **"app_header"**.

If you want to change maximum flash size then you must change another parameter:

1. In application *.ld (app_32MX795F512L.ld) file change LENGTH parameter

```
kseg0_program_mem    (rx) : ORIGIN = 0x9D001490,
LENGTH = (0x80000 - 0x1490 - 0x40)
```

Structure app_header has the following fields:

app_entry_addr – Start address for user application.

app_start_addr – Address from which bootloader should start to write the user application into flash memory (filled by the compiler).

programm_version – the software version number (filled by user)
date_of_compile – date of compile (filled by user)

reserved[4] – free fields, the user can use for their purposes

crc_binary_file – CRC16 binary file, (filled by the program "Remedy File Coder")

crc_old_binary_file – CRC16 binary file (used bootloader for recovery firmware)

crc_coded_file – CRC16 encoded binary file (filled by the program "Remedy File Coder")

mark_coded – flag of the encoded file or not (filled by the program "Remedy File Coder")

3. Quick start bootloader and user application

3.1 Download an application by bootloader

1. Build the MCU_Bootloader project
2. Connect to the board and download MCU_Bootloader project from MPLABX. See your board's Quick Start Guide for details on how connect and program your board from MPLABX.
3. Connect a serial cable to UART 1 (if needed) and open Serial Terminal. Use a connection speed of 115200 Baud. After pressing the reset button, the bootloader should start. The next screen should be displayed:

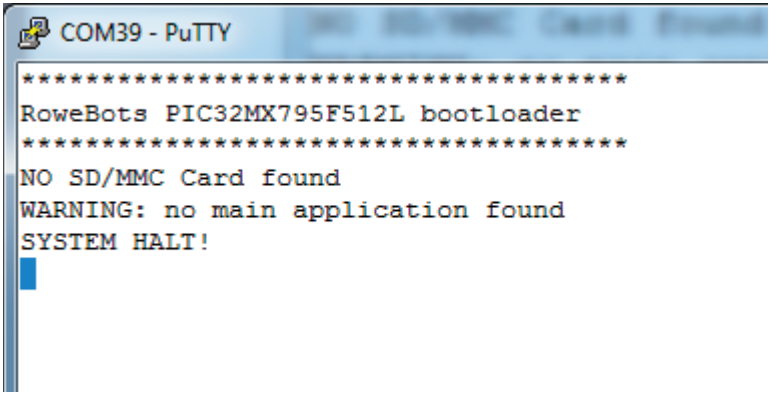


Figure f2
Bootloader starting screen

4. Build the user application project.
5. Prepare file for bootloader

Open the program "Remedy File Coder", select in user application project directory file *.*bin* and run. Remedy File Coder creates a file with extension *.*ocf*.

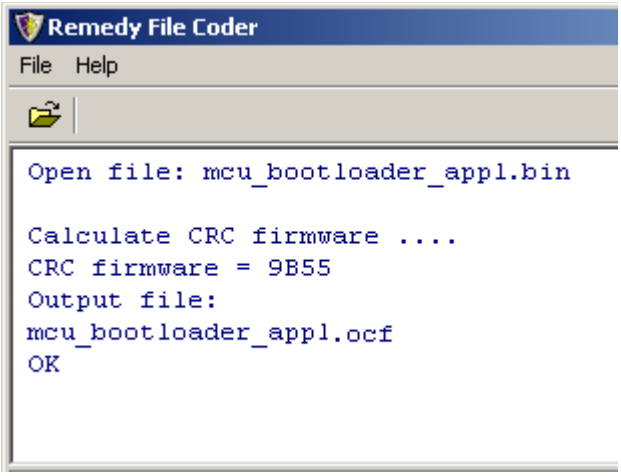


Figure f3
Prepare *.*ocf* file for bootloader

6. Write created file to SD card in folder *"/firmware"* and rename **.ocf* file to *fw.ocf*.

Figure f4
Write **.ocf* file to SD card

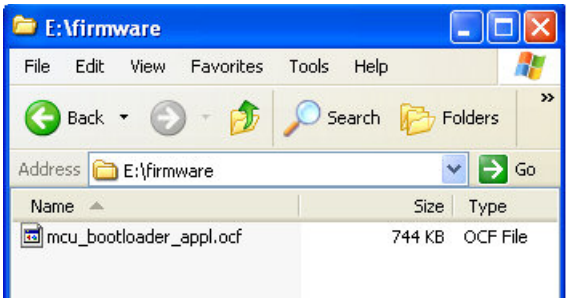
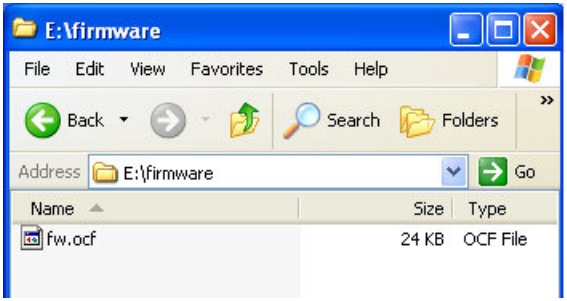
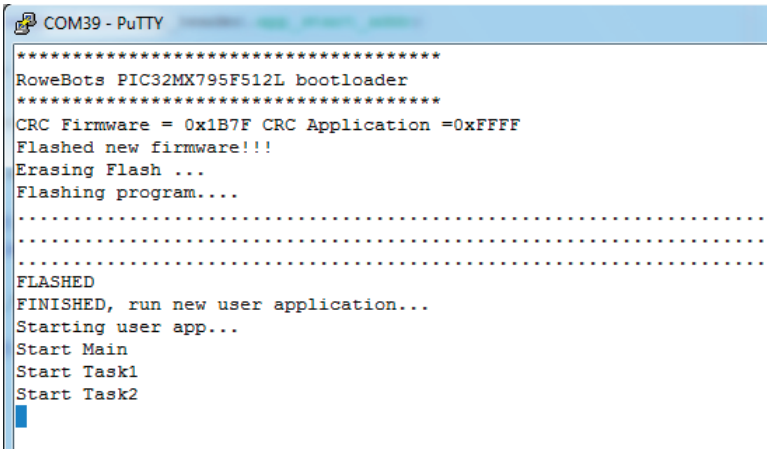


Figure f5
Rename **.ocf* file to *fw.ocf*



7. Plug the SD card into your hardware application and turn power on. The bootloader should now find the *fw.ocf* file and start to erase flash section and flashes user application and run.

Figure f6
Flash new firmware. Run new user application



4. Algorithm and modes of bootloader operation

4.1 Bootloader modes.

Optionally bootloader can operate in two modes:

- simple firmware update mode,
- recovery update firmware mode.

In simple mode the Bootloader checks the SD card contents for the firmware file and if the file is present on the SD card, programs it to MCU flash.

In recovery update mode the Bootloader check SD card for new firmware as in simple mode, and additionally, can program older version of firmware if the new firmware version failed. In this mode, the Bootloader is expecting to have 2 firmware at SD card: current (old) and new. More details will be described later in this document.

To switch on/off recovery update firmware mode, used define **USE_APPL_RECOVERY** in file bootloader.h in application MCU bootloader.

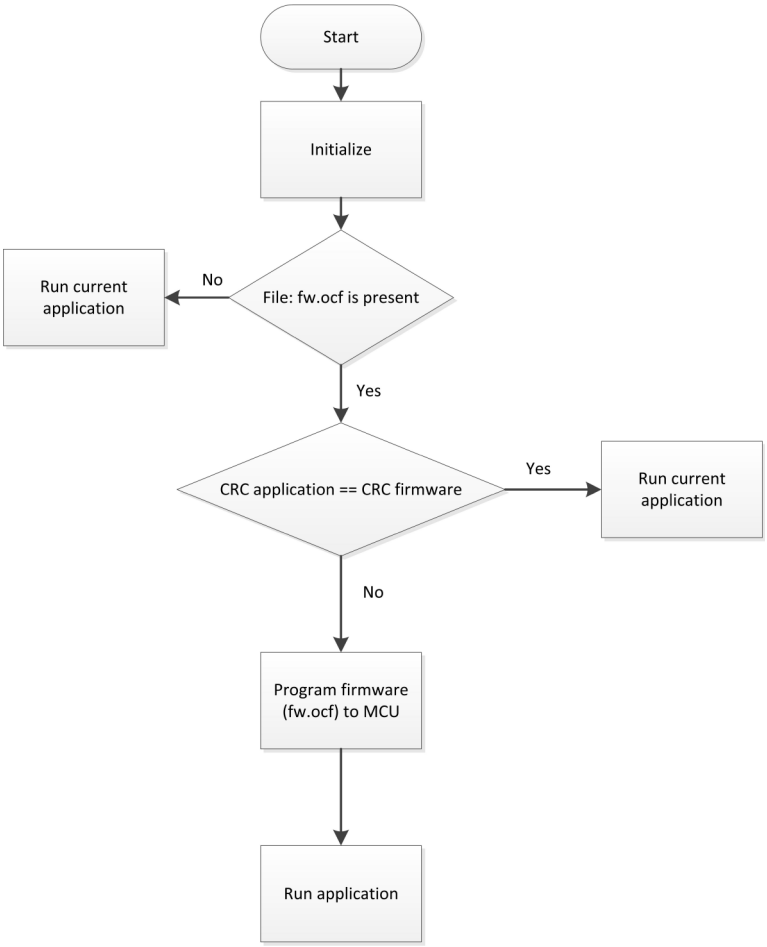
4.1.1 Simple firmware update mode

After the MCU is powered up, the bootloader application will start first. It will check the SD card and try to find a file with new firmware to program into the FLASH memory.

If this file exists, and if the CRC firmware check is different from the CRC for the existing programmed application, the bootloader calculates and checks the CRC16 of this file and if it matches the expected CRC, the new firmware is programmed into the internal MCU flash and the bootloader tries to start the application. If the bootloader detects that the application is identical with firmware fw.ocf, the bootloader starts the existing application without programming.

The algorithm for this mode of operation is shown in figure f7.

Figure f7
Bootloader Flow diagram without recovery mode



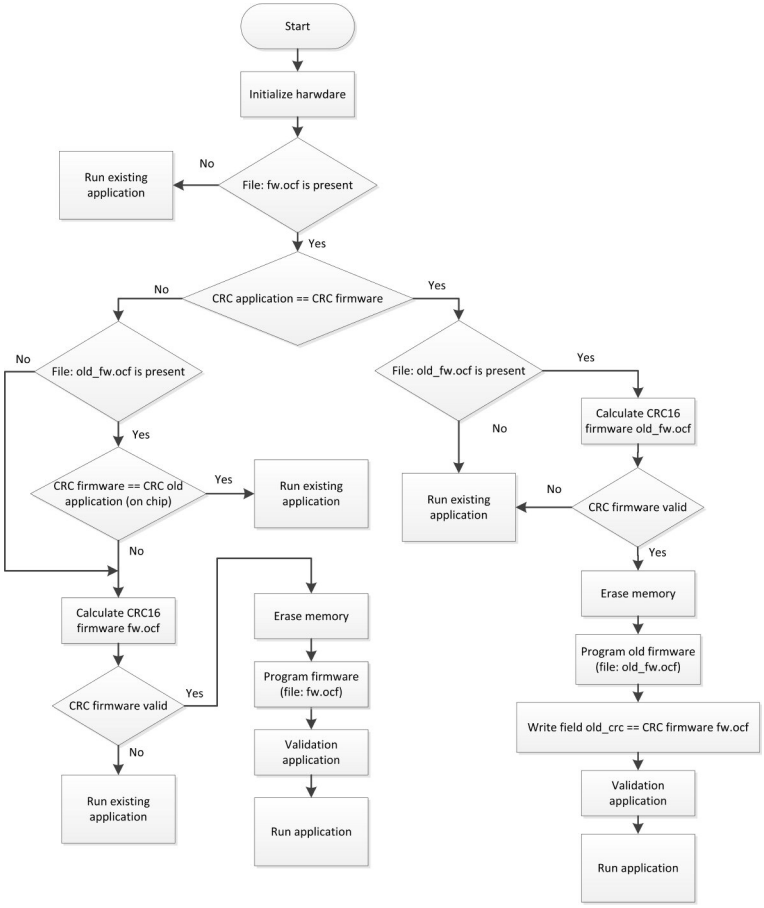
4.1.2 Recovery firmware update mode

The algorithm for this operation mode is based on an approach of new firmware file renaming by the User application in the case of successful startup after programming. The firmware fw.ocf must be renamed to old_fw.ocf,

If new User application can't start for some reason and so doesn't rename the new firmware file fw.ocf to old_fw.ocf, then Bootloader can reload old stable firmware during next power on.

The algorithm for this mode of operation is shown in figure f8.

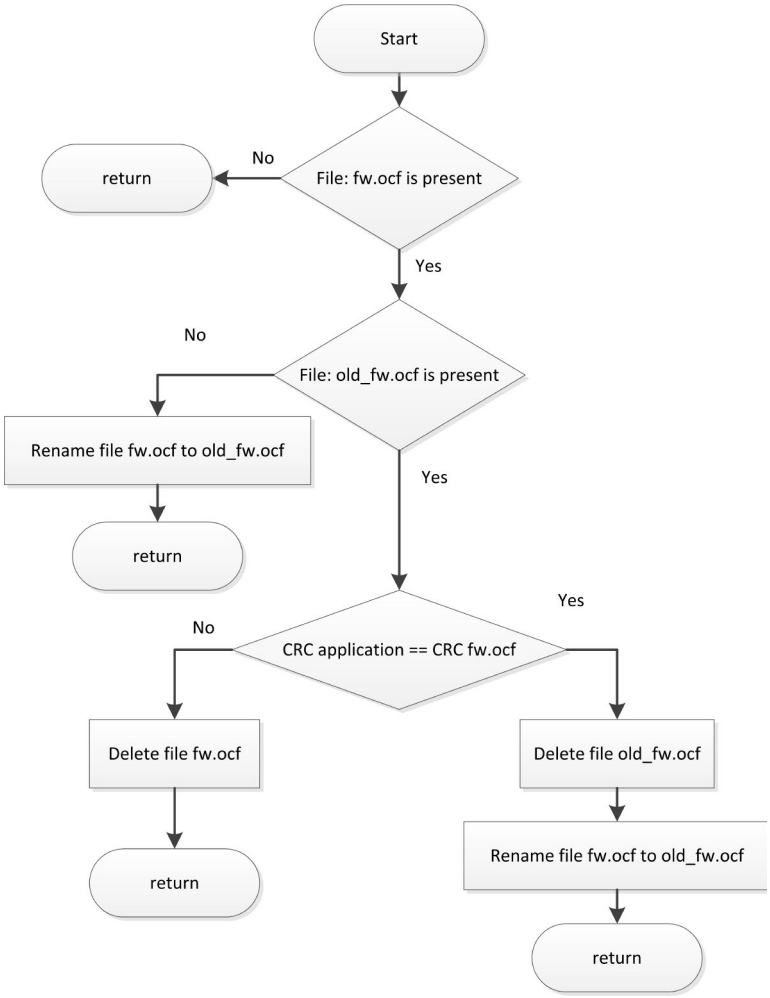
Figure f8
Bootloader Flow diagram with recovery mode



The user application should implement the algorithm, described below.

The main idea (as mentioned above) is to rename current firmware file on the SD card – *fw.ocf* to *old_fw.ocf*. Note that the CRC is also updated by the bootloader if the new firmware fails to run and change the file name.

Figure f9
Application Flow diagram with recovery mode



5. Application update example

5.1 Preparing firmware binary files

To prepare MPLAB output file for loading with booloader you need make this steps:

1. Copy application *.*hex* file from *{$app_dir}\ dist\ ESK\ production*.*hex* to *{demo_dir}\bootloader\convert* .
2. Rename application *.*hex* file to *fw.hex* .
3. Run *convert.cmd* from *\convert* directory. After script finished - you got fw.bin in *\convert* directory.
4. Run Remedy Files *Coder.exe* and open *fw.bin* .
5. Generate *fw.ocf* with Remedy Files Coder.

5.2 Programming binary files

1. In project **mcu_bootloader_appl_recovery** open file main.c and change the target IP-address to the address from your network (see structure **etherinit**).
2. Compile **mcu_bootloader_appl_recovery** project **Run->Clean and Build Project.**
3. Prepare binary files before programming it as defined in 5.1.
4. Program MCU Flash as described above.
5. Check that **mcu_bootloader_appl_recovery** project started successfully. You can see in COM-port terminal:

```
  ************************************
  RoweBots PIC32MX795F512L bootloader
  ************************************
  CRC Firmware = 0x7BA CRC Application =0xFFFF
  Flashed new firmware!!!
  Erasing Flash ...
  Flashing program....
.................................................................
.................................................................
  FLASHED
  FINISHED, run new user application...
  Starting user app...
  Start Main
  mcu_bootloader_appl_recovery #0
  fsys-fatfs: 1.5.4
  starting fsys main loop
  ..FAT File server is now created..
  SD card driver has number 0
```

```
..Attempting to mount it...
..done fsys initialization..
file /firmware/fw.ocf found rename it on /firmware/old_fw.ocf

--- Start tcpd ---
tcpd: 1.2.2
tcpd: device = 1, vector = 48, level = 81, devname = le
init localloop
init le0
telnetd: v1.2

telnetd started!

--- Start TFTP server ---
mkdir /dev/sd/disc0/firmware exist
tftpd: version 1.1
tftpd: work directory = /dev/sd/disc0/firmware
```

6. In the project **mcu_bootloader_appl_recovery** open the file main.c and change in function Main() number to “2”:

   xprintf("mcu_bootloader_appl_recovery #2\n");

7. Compile **mcu_bootloader_appl_recovery** project **Run->Clean and Build Project.**

8. Prepare binary files before programming it as defined in 5.1

9. Open **mcu_bootloader_appl_recovery** project directory:

cd **{Unison_install_dir}\demo\bootloader \ mcu_bootloader_appl_recovery**

   Upload new application .hex file to target:

   upload.cmd ***_target-IP_***

10. Connect to target by telnet protocol:

    login: admin

    password: RoweBots#1

11. Open firmware directory:

    cd firmware

12. Check new file fw.ocf availability:

    ls -l

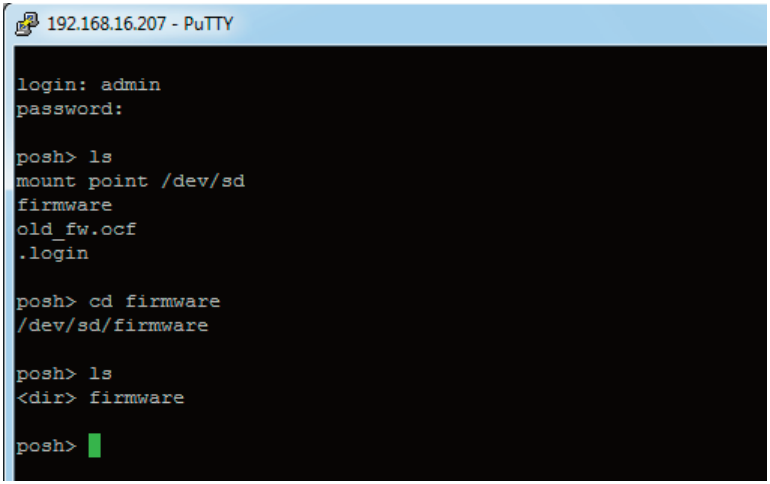
13. Reboot the target:

    appreboot

Check that new mcu_bootloader_appl_recovery project started successfully. You can see in COM-port terminal:

```
  ************************************
  RoweBots PIC32MX795F512L bootloader
  ************************************
  CRC Firmware = 0x9E42 CRC Application =0x7BA
  Flashed firmware!!!
  Erasing Flash ...
  Flashing program....
.................................................................
.................................................................
  Finished
  Starting user app...
  Start Main
  mcu_bootloader_appl_recovery #2
  fsys-fatfs: 1.5.4
  starting fsys main loop
  ..FAT File server is now created..
  SD card driver has number 0

  ..Attempting to mount it...
  ..done fsys initialization..
  CRC appl=0x9E42, CRC firmware =0x9E42
  Delete /firmware/old_fw.ocf
  file /firmware/fw.ocf found, rename it to /firmware/old_fw.ocf

  --- Start tcpd ---
  tcpd: 1.2.2
  tcpd: device = 1, vector = 48, level = 81, devname = le
  init localloop
  init le0
  telnetd: v1.2

  telnetd started!

  --- Start TFTP server ---
  mkdir /dev/sd/disc0/firmware exist
  tftpd: version 1.1
  tftpd: work directory = /dev/sd/disc0/firmware
```

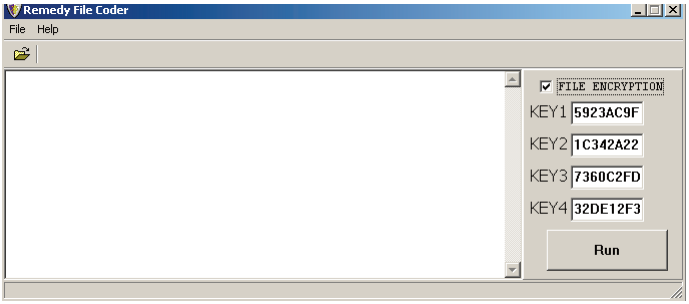
Figure f10
Telnet user application session



6. Firmware encryption / decryption

Optionally, you can encrypt/decrypt firmware. Firmware encryption implemented with using of the “Raiden encryption algorithm” with 128-bits key. To create an encrypted firmware, it is necessary to select "FILE ENCRYPTION" check box in “Remedy File Coder” tool and specify your unique 128 bit key. See figure f11.

Figure f11
The PC “Remedy File Coder” application



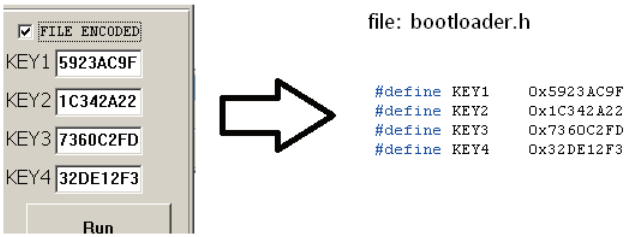
To simplify user interface of “Remedy File Coder” tool, 128 bit key was divided to four parts:
KEY1, KEY2, KEY3, KEY4.
And meaning of these fields - are follow:

Figure f12
Mapping 128 bits key encryption with keys of panel “Remedy File Coder” application

128	96	64	32	0
KEY4	KEY3	KEY2	KEY1	
32DE12F3	7360C2FD	1C342A22	5923AC9F	

The same key should be specified in the mcu_bootloader project - file bootloader.h, defines KEY1,KEY2,KEY3 and KEY4

Figure f13
Compliance with the keys in the program prescribed by“Remedy File Coder” with the keys in the bootloader



To enable/disable decryption support, should be used define USE_DECRYPTION in bootloader.h:

#define USE_DECRYPTION 0 //1 – enable decryption, 0 – disable

Bootloader can automatically detects is it necessary or not to decrypt firmware. So it is not mandatory to use encrypted firmware in this mode.

7. Watchdog Timer

Optionally, you can enable watchdog in before start user application by setting

```
#define WDT_ENABLE 1 //0 to disable WD timer
```

This option used to guarantee that the correct application was loaded into flash. Normal application after start (delay not more then 2 sec) must restart (or disable) watchdog timer. If application will not restart (or disable) watchdog timer - bootloader will reboot application and restore last successful firmware from a backup copy, if algorithm with revert to old firmware is enabled (#define USE_APPL_RECOVERY 1).

8. Device configuration bits

The PIC32 device family has programmable nonvolatile device configuration words that define the behavior of the device. For more information on the device configuration registers, refer the device data sheet.

Note: The bootloader does not erase or write the device Configuration words while programming the new application firmware. This is because the device Configuration words settings are shared by both the bootloader and the user application. Therefore, it is recommended to have the same configuration settings in both the application and the bootloader code.

Default configuration settings:

```
#pragma config DEBUG    = OFF       // Background Debugger disabled
#pragma config FPLLMUL  = MUL_18    // PLL Multiplier
#pragma config FPLLIDIV = DIV_2    // PLL Input Divider
#pragma config FPLLODIV = DIV_1    // PLL Output Divider

#pragma config POSCMOD  = HS      // Primary Oscillator Mode: High
Speed xtal
#pragma config FNOSC    = PRIPLL  // Oscillator Selection: Primary
oscillator  w/ PLL
#pragma config FPBDIV   = DIV_2  // Peripheral Bus Clock: Divide by 1
#pragma config BWP      = OFF    // Boot write protect: OFF

#pragma config FWDTEN   = OFF      // WD timer: OFF
#pragma config ICESEL   = ICS_PGx1 // ICE pins are shared with PGC1,
PGD1

#pragma config FCKSM   = CSDCMD  // Clock Switching Enabled, Clock
Monitoring Disabled
#pragma config OSCIOFNC = OFF   // CLKO Disabled
#pragma config IESO     = OFF   // Internal/External Switch-over dis-
abled
#pragma config FSOSCEN  = ON   // Secondary Oscillator disabled
#pragma config CP       = OFF  // Code Protect disabled
#pragma config PWP     = OFF  // Program Flash Write Protect disabled
```

Additional user can enable Ethernet and USB controllers by set parameters:

```
#define ENA_ETH 1
#define ENA_USB 1
```

9. Appendix
How to create a user application demo for bootloader

1. Create demo application using File->New Project…

1.1 Select project type: Microchip Embeded -> Standalone Project

1.2 Select CPU type (PIC32MX795F512L)

1.3 Select hardware debugger tool (example Real ICE)

1.4 Select Plugin Board (none)

1.5 Select Compiler tool (XC32)

1.6. Enter Project name, project location and project folder

2. Modify project settings, by adding include search path, library path, and other project options - see picture below:

Figure f14
Project Settings

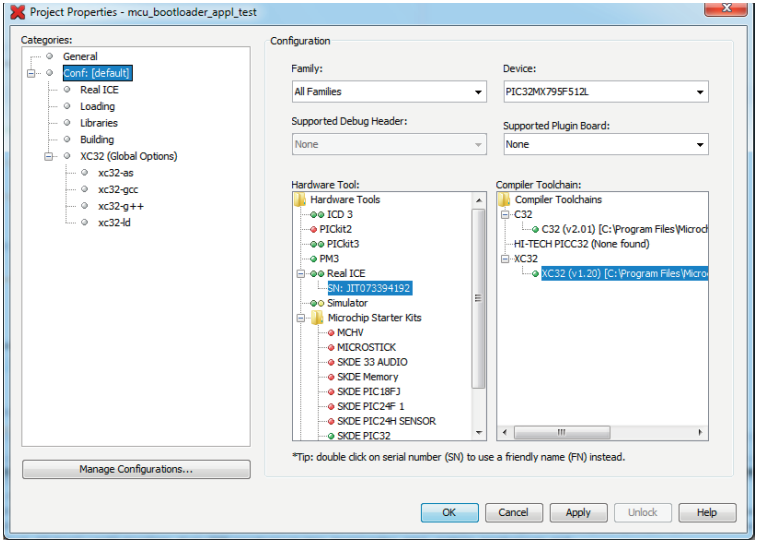


Figure f15
Add library files which

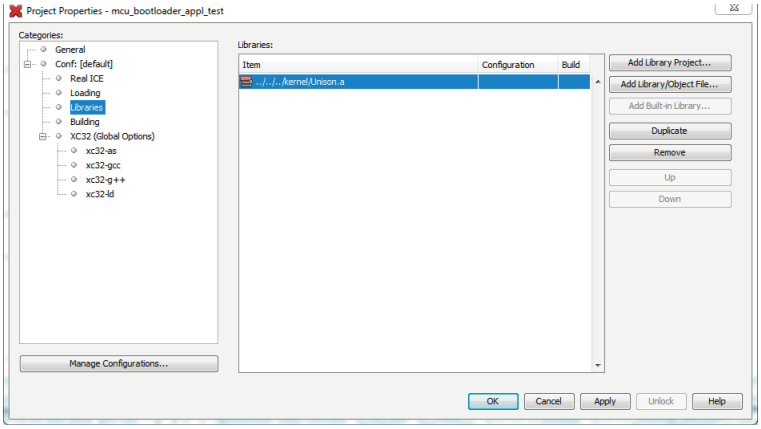
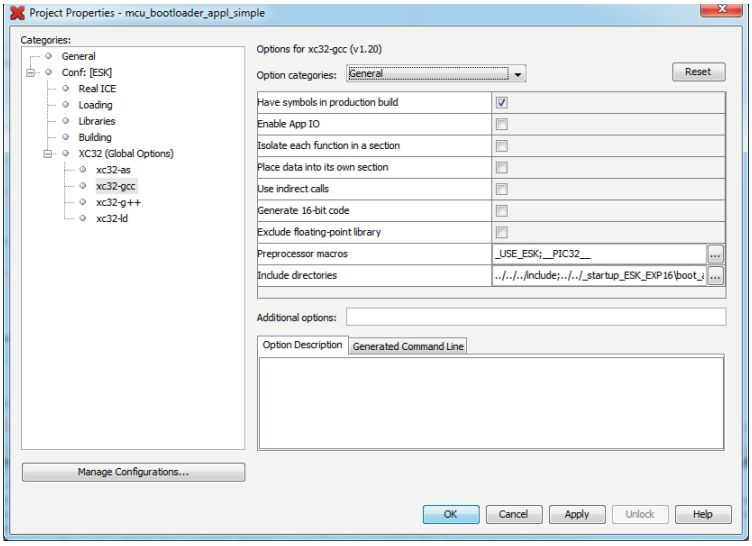


Figure f16
Set Includes path and Preprocessor macros.



Includes path. Example:

```
../../../include
../../_startup_ESK_EXP16\boot_app\
```

Preprocessor macros. Example:

```
_USE_ESK
__PIC32__
```

Add files from template to project:

1. Add linker script *.ld from {demo_dir}_startup_ESK_EXP16\boot_app\
2. Add setup.c and apploader.c from {demo_dir}_startup_ESK_EXP16\boot_app\

Add local project files (main.c, malloc.c, etc.)

Now you can build project by executing command Run->Clean and Build Project

After build, MPLABX will create file *.hex file in {$Proj_directory}\dist\default\production\ directory